2303A51242

Batch-05

Assignment-10.1

Task-1 :Syntax and Logic Errors

Prompt: #You are a Python code reviewer.

#1. Identify all syntax and logical errors in the following Python code.

#2. Correct the errors.

#3. Provide the fully corrected runnable code.

#4. Clearly explain each fix you made.

Calculate average score of a student

"""def calc_average(marks):

    total = 0

    for m in marks:

        total += m

    average = total / len(marks)

    return avrage   # Typo here

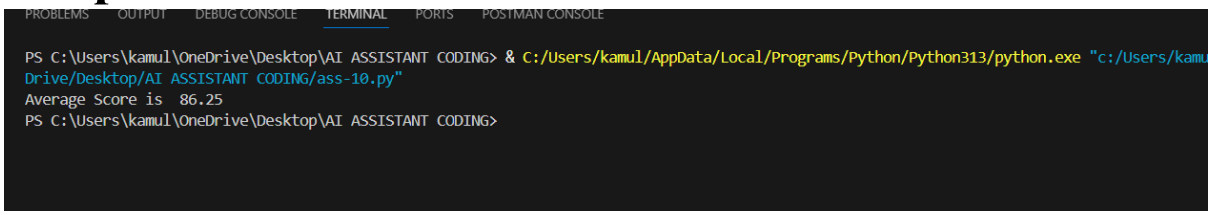
marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)"""

Code:

```
18  def calc_average(marks):
19      total = 0
20      for m in marks:
21          total += m
22      average = total / len(marks)
23      return average   # Fixed typo here
24  marks = [85, 90, 78, 92]
25  print("Average Score is ", calc_average(marks))  # Added closing parenthesis here
```

Output:



Explanation: 1. In the original code, there was a typo in the return statement of the `calc_average` function. The variable `average` was misspelled as `avrage`, which would have caused a NameError when the function is called. I corrected this by changing `avrage` to `average`.

2. The original code was missing a closing parenthesis at the end of the `print` statement, which would have caused a SyntaxError. I added the closing parenthesis to ensure the code runs correctly.

Task-2: PEP 8 Compliance

Prompt: Refactor the following Python code to strictly follow PEP 8 style guidelines.

#1. Fix spacing, indentation, and formatting.

#2. Improve function and variable naming if necessary.

#3. Ensure the final version is clean and readable.

"""

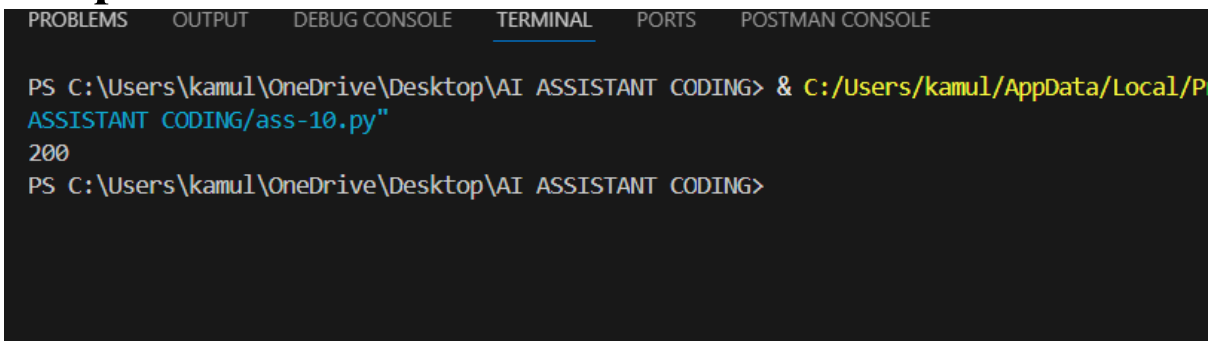
def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))"""

Code:

```
"""
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))"""
def area_of_rectangle(length, breadth):
    """Calculate the area of a rectangle."""
    return length * breadth
# Example usage
print(area_of_rectangle(10, 20))
```

Output:



Explanation: The function name has been changed to `area_of_rectangle` for clarity, and the parameters have been renamed to `length` and `breadth` for better readability. The code is now properly formatted with appropriate spacing and indentation, following PEP 8 guidelines.

Task -3: Readability Enhancement

Prompt: Improve the readability of the following Python code without changing its logic.

1. Use descriptive function and variable names.

2. Add meaningful inline comments.

3. Improve formatting and structure.

4. Keep the logic exactly the same.

```
def c(x,y):
 return x*y/100
a=200
b=15
print(c(a,b))
```

Code:

```python
#4. Keep the logic exactly the same.
def calculate_percentage(value, percentage):
    """Calculate the percentage of a given value."""
    return value * percentage / 100
a = 200
b = 15
print(calculate_percentage(a, b))
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c
ASSISTANT CODING/ass-10.py"
30.0
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The function `calculate_percentage` takes two parameters, `value` and `percentage`, and returns the calculated percentage of the value. The variable names are descriptive, and an inline comment explains the purpose of the function. The code is formatted for better readability while maintaining the same logic.

Task-4: Refactoring for Maintainability

Prompt: Refactor the following Python code to improve maintainability.

1. Remove repetitive code.

2. Create reusable functions.

3. Make the code modular and scalable.

4. Keep the output behavior the same.

students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])

Code:

```
from typing import List
def welcome_students(students: List[str]) -> None:
    """Welcome each student in the list."""
    for student in students:
        print("Welcome", student)
# Example usage
students = ["Alice", "Bob", "Charlie"]
welcome_students(students)
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/
ASSISTANT CODING/ass-10.py"
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The refactored code defines a function `welcome_students` that takes a list of student names as an argument. The function iterates through the list and prints a welcome message for each student. This eliminates the repetitive code and makes the function reusable for any list of students, improving maintainability and scalability.

Task-5: Performance Optimization

Prompt: Optimize the following Python code for better performance.

1. Make it faster and more memory efficient.

2. Use list comprehensions or other Pythonic optimizations.

3. Keep the output the same.

4. Briefly explain why your version is faster.

Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

    squares.append(n**2)

print(len(squares))

Code:

```
#3. Keep the output the same.
#4. Briefly explain why your version is faster.
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
# Optimized version using list comprehension
squares = [n**2 for n in range(1, 1000000)]
print(len(squares))
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Lo
ASSISTANT CODING/ass-10.py"
999999
999999
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The optimized version uses a list comprehension to generate the list of squares in a single line, which is more efficient than appending to a list in a loop. List comprehensions are optimized in Python and can be significantly faster than using a for loop with append, especially for large lists, as they reduce the overhead of method calls and allow for better memory management.

Task -6 :Complexity Reduction

Prompt: Simplify the following Python function to reduce complexity.

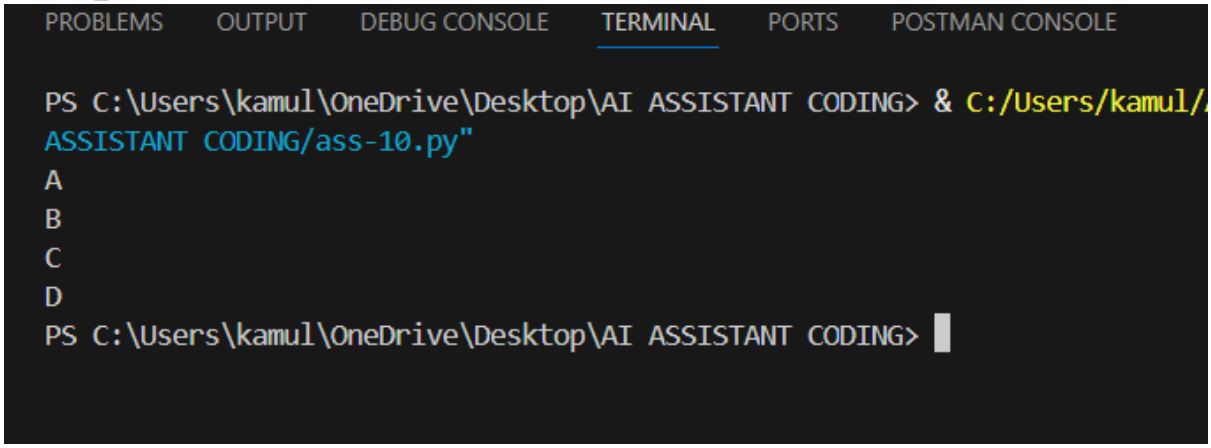
1. Replace deeply nested if-else statements.
2. Use cleaner logic such as elif or mapping.
3. Keep the grading logic exactly the same.
4. Provide the improved version of the code.

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

Code:

```python
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
#example usage
print(grade(95))  # Output: A
print(grade(85))  # Output: B
print(grade(75))  # Output: C
print(grade(65))  # Output: D
```

Output:



Explanation: The original function had multiple nested if-else statements, which made it more complex and harder to read. The improved version uses elif statements to simplify the logic and reduce nesting, while maintaining the same grading logic. This makes the code cleaner and easier to understand.